

Deep Learning para Business Analytics

Tema B — Sistemas sobre el agente

RAG, MCP, multi-agente, adaptación y el presupuesto completo.

Eduardo C. Garrido-Merchán

Profesor Propio Adjunto, Métodos Cuantitativos, ICADE

ecgarrido@comillas.edu



Madrid, 2026



herramientas del tema

Cinco sistemas que se construyen encima del agente.

Cinco sistemas que se construyen encima del agente.



Coste y esfuerzo crecientes → probad siempre de izquierda a derecha

Cómo se lee esta hoja de ruta.

Cinco sistemas que se construyen encima del agente.



Coste y esfuerzo crecientes → probad siempre de izquierda a derecha

El orden importa. Los cinco bloques están ordenados por coste y por esfuerzo. Casi todos los proyectos que fracasan empezaron por el cuarto.

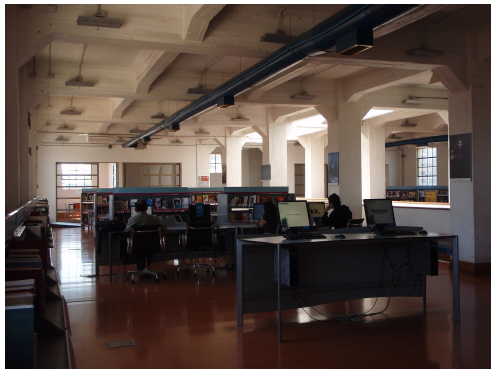
La pregunta que los separa. ¿Al modelo le falta *información* (RAG), le faltan *acciones* (MCP), le falta *capacidad de trabajo* (multi-agente) o le falta *criterio* (adaptación)?

El quinto no es opcional. Sin un ciclo de medida, los otros cuatro son decoración: nadie sabe si el cambio mejoró algo.

IDEA CLAVE. Este tema cierra con el presupuesto completo, que es el entregable del proyecto final.

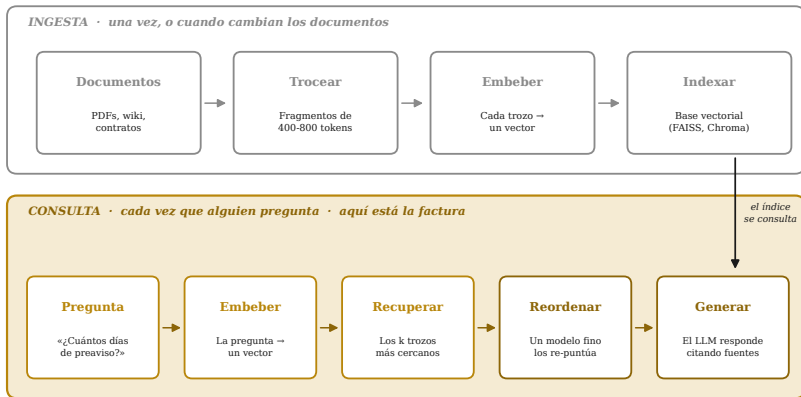
Parte 1 / 6

RAG: darle acceso a vuestros documentos



RAG completo: una fase que se hace una vez y otra que se paga siempre.

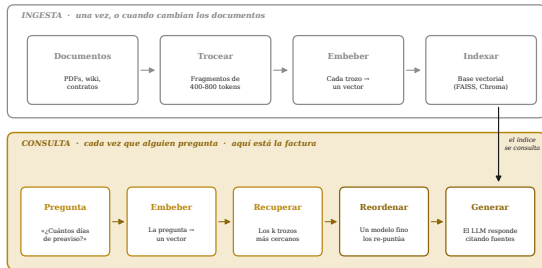
RAG completo: una fase que se hace una vez y otra que se paga en cada consulta.



El 90 % de los tutoriales acaban en «Recuperar». El reordenado y las citas son lo que separa un juguete de algo que se puede enseñar a un cliente.

Por qué RAG y no meter los documentos en el prompt.

RAG completo: una fase que se hace una vez y otra que se paga en cada consulta.



El 90 % de los tutoriales acaban en «Recuperar». El reordenado y las citas son lo que separa un juguete de algo que se puede enseñar a un cliente.

El problema. Vuestra empresa tiene 4.000 documentos internos. Son decenas de millones de tokens: no caben, y si cupieran costarían cientos de euros por pregunta.

La idea. Recuperar sólo los cinco fragmentos relevantes y pasarle esos. El modelo responde con lo que tiene delante.

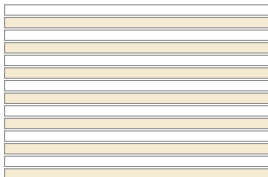
El efecto colateral valioso. Como el fragmento viene de un documento concreto, el sistema puede **citar la fuente**. Sin cita, ningún departamento de riesgos os deja salir a producción.

IDEA CLAVE. RAG convierte «el modelo sabe» en «el modelo tiene delante», que es auditable.

El tamaño del fragmento decide qué puede responder el sistema.

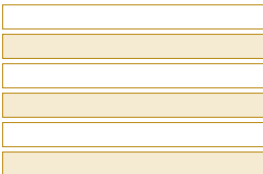
El tamaño del fragmento decide qué puede responder el sistema.

Trozos cortos (150 tok)



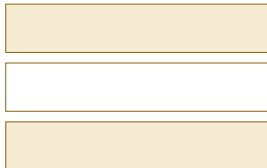
Recupera muy preciso.
Pierde el contexto:
el «él» de un párrafo
ya no sabe a quién
se refiere.

Trozos medios (500 tok)



El punto dulce habitual.
Un apartado completo
con su encabezado.
Solapamiento del 10-15 %
entre trozos.

Trozos largos (2.000 tok)



Mucho contexto, pero
cada trozo trae ruido.
Pagáis tokens por
párrafos que no hacían
falta.

Trocear: el parámetro que más se descuida.

El tamaño del fragmento decide qué puede responder el sistema.



El compromiso. Trozos cortos recuperan con precisión pero pierden el contexto. Trozos largos traen contexto y también ruido que pagáis en tokens.

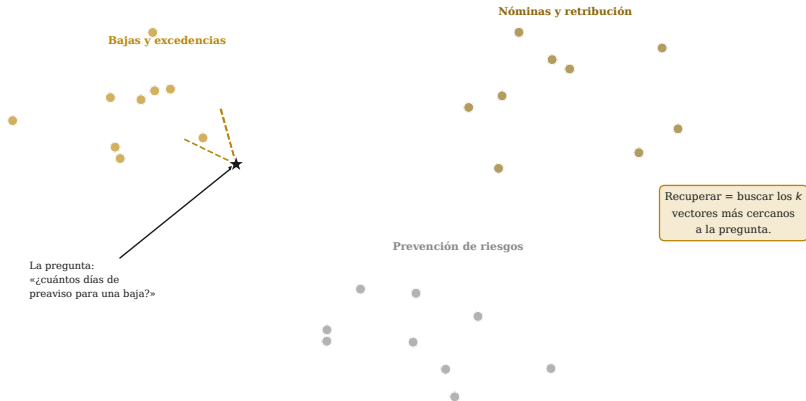
Punto de partida razonable. Entre 400 y 800 tokens, con un solapamiento del 10 al 15 % entre trozos consecutivos para no cortar una idea por la mitad.

Mejor que afinar el tamaño. Trocear respetando la estructura del documento: por apartado, por artículo del contrato, por fila de la tabla. Casi siempre gana a cualquier número fijo.

IDEA CLAVE. Si un fragmento no se entiende leído solo, el modelo tampoco lo entenderá.

Un embedding coloca cada fragmento en un mapa donde la cercanía es significativo.

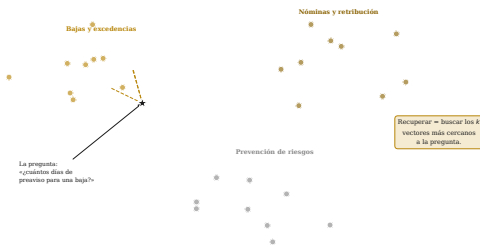
Un embedding coloca cada fragmento en un mapa donde la cercanía es significado.



El mapa real tiene entre 384 y 3.072 dimensiones; aquí lo vemos en dos. La distancia se mide con similitud coseno.

Similitud coseno: la cuenta que hace la recuperación.

Un embedding coloca cada fragmento en un mapa donde la cercanía es significado.



El mapa real tiene entre 384 y 3.072 dimensiones; aquí lo vemos en dos. La distancia se mide con similitud coseno.

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}$$

$\mathbf{q}$ vector de la pregunta $\mathbf{d}$ vector de un fragmento

Resultado en $[-1, 1]$: 1 es misma dirección, 0 sin relación.

Por qué el coseno. Mide el ángulo, no la magnitud: un documento largo y uno corto sobre lo mismo apuntan en la misma dirección.

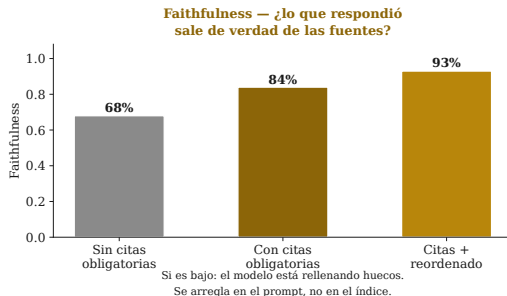
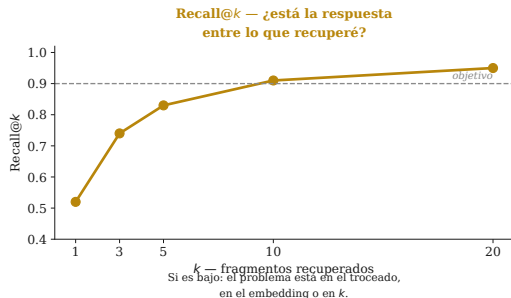
EJEMPLO A MANO: A mano, en dos dimensiones

Con $\mathbf{q} = (3, 4)$ y $\mathbf{d} = (6, 8)$: producto escalar = 50, normas 5 y 10, luego $\text{sim} = 50/50 = 1$.

IDEA CLAVE. La única fórmula del tema que entra en el examen.

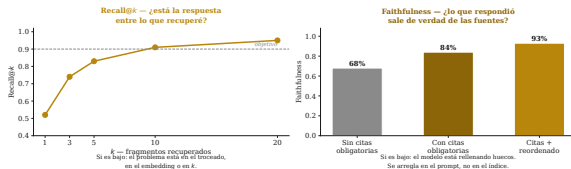
Dos métricas, dos fallos distintos, dos arreglos distintos.

Dos métricas, dos fallos distintos, dos arreglos distintos.



Cómo se mide un RAG sin autoengañarse.

Dos métricas, dos fallos distintos, dos arreglos distintos.



Recall@k. De las preguntas de vuestro conjunto de prueba, ¿en qué porcentaje el fragmento correcto estaba entre los k recuperados? Mide el *buscador*.

Faithfulness. De las afirmaciones de la respuesta, ¿cuántas se sostienen en los fragmentos que se le pasaron? Mide el *generador*.

Por qué hacen falta las dos. Un sistema puede recuperar perfectamente y aun así inventar. Y puede responder cosas ciertas que no estaban en las fuentes: eso también es un fallo.

IDEA CLAVE. Treinta preguntas reales con su respuesta acordada bastan para empezar. Sin ese conjunto, cualquier mejora es una impresión.

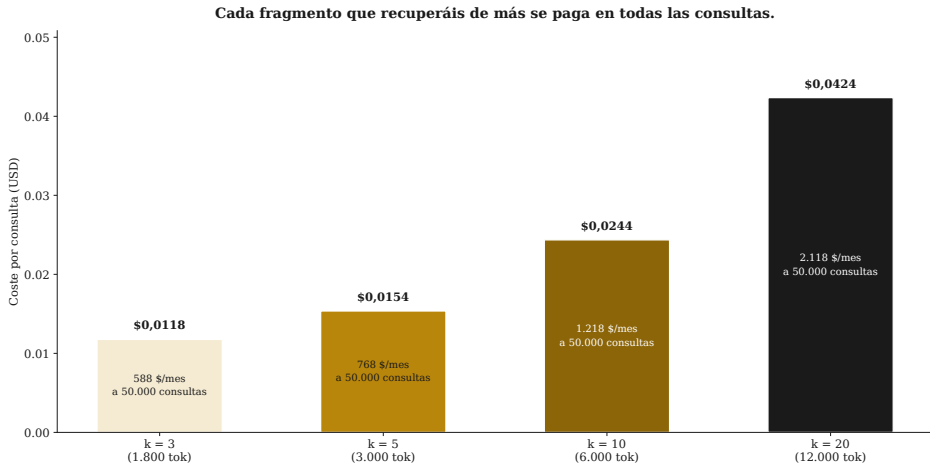
Cuando un RAG responde mal, el fallo está en uno de cuatro sitios.

Cuando un RAG responde mal, el fallo está en uno de cuatro sitios.

El documento no está indexado	Está indexado pero no se recupera	Se recupera pero queda enterrado	Se recupera y aun así responde mal
Síntoma: inventa. Diagnóstico: buscad el texto a mano en el índice.	Síntoma: Recall@k bajo. Diagnóstico: buscad el trozo con la pregunta exacta.	Síntoma: está en el puesto 18 de 20 fragmentos.	Síntoma: faithfulness baja. El dato estaba delante.
Arreglo: revisad la ingesta. ¿Se leyó bien ese PDF?	Arreglo: troceado, modelo de embedding, subir k.	Arreglo: reordenado. Sube lo relevante a los primeros puestos.	Arreglo: prompt. Exigid cita literal y permitid «no lo sé».

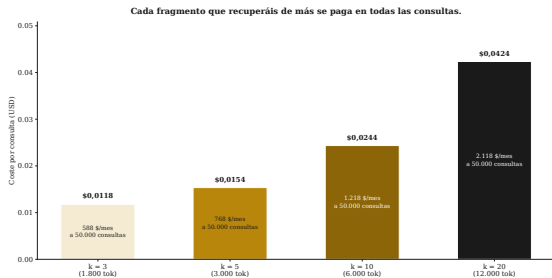
Los cuatro se diagnostican en ese orden, y cada uno tiene un arreglo distinto. Cambiar de modelo casi nunca arregla ninguno.

Cada fragmento que recuperáis de más se paga en todas las consultas.



Sonnet 5, fragmentos de 600 tokens, 400 de respuesta, system prompt cacheado. Precios a 2026-10-02.

El coste de un RAG, partida a partida.



Sonnet 5, fragmentos de 600 tokens, 400 de respuesta, system prompt cachado. Precios a 2026-10-02.

Una vez. Embeber el corpus. Un millón de tokens de documentos cuesta céntimos con un modelo de embeddings: es la partida que nadie debería discutir.

Cada consulta. Embeber la pregunta (despreciable) y, sobre todo, los k fragmentos que inyectáis en el prompt. Ahí está el 90 % de la factura.

Mensual fijo. La base vectorial alojada, si no la ponéis vosotros.

La palanca. Subir k de 5 a 20 cuadruplica los tokens de contexto y sube el recall unos 12 puntos. Reordenar con un modelo barato os da ese recall con k pequeño.

IDEA CLAVE. El reordenado es el cambio con mejor relación calidad/coste de todo el pipeline.

Qué está pasando ya en las empresas de vuestro sector.

Cinco Días · Empresas

CincoDías

Las eléctricas montan buscadores internos sobre su normativa

El patrón se repite en banca, energía y seguros: indexar la normativa interna y dejar que el empleado pregunte en lenguaje natural, con la cita al documento original.

Expansión · Tecnología

Expansión

El 80 % de los proyectos de IA se quedan en la prueba piloto

La causa que más se repite no es técnica: no hay forma acordada de medir si el sistema funciona, así que nadie se atreve a ponerlo delante de un cliente.

Lo que separa el piloto del producto

Un conjunto de evaluación con casos reales y respuesta acordada.

Una métrica por cada tipo de fallo, medida antes y después.

Un responsable que decide si el número es suficiente para salir.

Parte 2 / 6

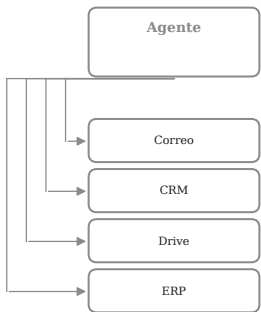
MCP: darle acceso a vuestras herramientas



MCP: un enchufe estándar entre el agente y vuestros sistemas.

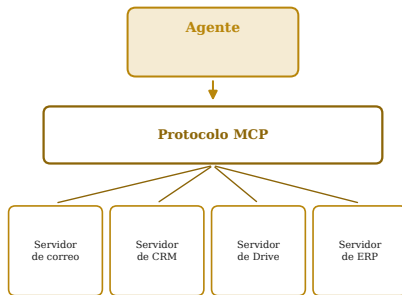
MCP: un enchufe estándar entre el agente y vuestros sistemas.

Antes: un conector a medida por sistema



4 sistemas = 4 integraciones
que mantener

Con MCP: un protocolo, servidores intercambiables

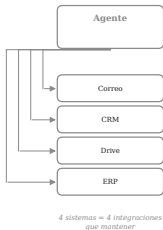


Cualquier servidor MCP que exista vale ya,
sin escribir integración.

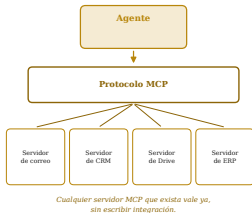
Qué problema resuelve el protocolo.

MCP: un enchufe estándar entre el agente y vuestros sistemas.

Antes: un conector a medida por sistema



Con MCP: un protocolo, servidores intercambiables



Antes. Cada equipo escribía su propio puente entre su agente y cada sistema. Con cuatro sistemas y tres agentes, doce integraciones que mantener.

Con MCP. El sistema expone un *servidor* que declara qué sabe hacer. Cualquier agente que hable el protocolo lo usa sin escribir código.

Las tres cosas que expone un servidor.

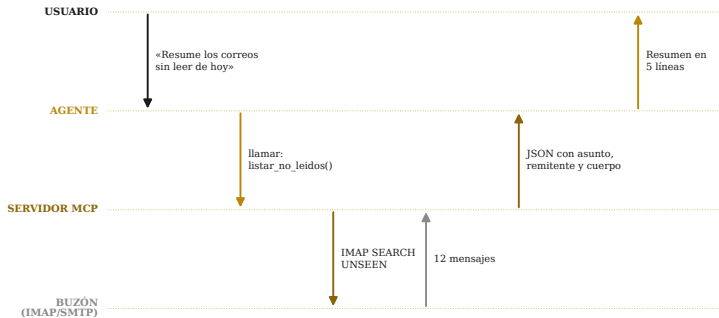
Herramientas (acciones que ejecuta), *recursos* (datos que puede leer) y *prompts* (plantillas preparadas).

IDEA CLAVE. Lo valioso es el ecosistema: hay servidores ya escritos para Drive, GitHub, Slack, Postgres o correo. Conectar cuesta minutos.

Un servidor MCP de correo, de principio a fin.

Un servidor MCP de correo, de principio a fin.

Las credenciales viven en el servidor MCP, nunca en el prompt ni en el contexto del modelo.

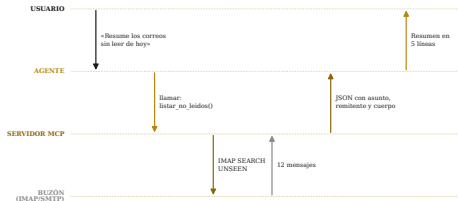


Leer es reversible y puede ir solo. Enviar es irreversible: ese método se configura como «pedir confirmación».

El caso del correo, con sus riesgos encima de la mesa.

Un servidor MCP de correo, de principio a fin.

Las credenciales viven en el servidor MCP, nunca en el prompt ni en el contexto del modelo.



Leer es reversible y puede ir solo. Enviar es irreversible: ese método se configura como «pedir confirmación».

Qué expone el servidor. `listar_no_leidos()`, `leer(id)`, `buscar(consulta)`, `redactar_borrador(...)`, `enviar(...)`.

Dónde viven las credenciales. En el servidor MCP, nunca en el prompt ni en el contexto del modelo. El modelo pide «enviar»; quien tiene la contraseña es el servidor.

Permisos por método. Los cuatro primeros van solos. `enviar` pide confirmación siempre, porque un correo a un cliente no se deshace.

El riesgo nuevo: inyección de prompt. Un correo entrante puede contener texto diseñado para dar órdenes al agente que lo lea. Si el agente puede leer correo y enviarlo, un desconocido acaba de adquirir capacidad de escribir en vuestro nombre.

IDEA CLAVE. Leer datos no fiables y poder actuar sobre el mundo es la combinación peligrosa. Separad las dos cosas o poned un humano entre ellas.

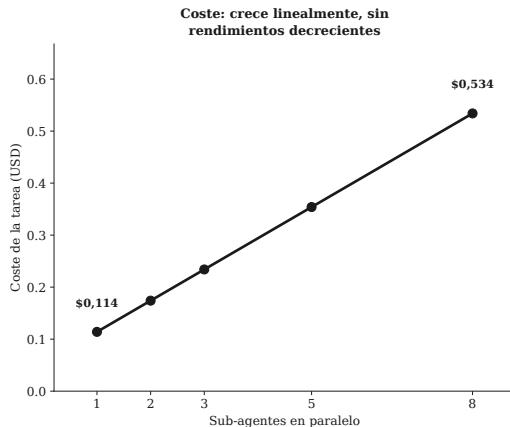
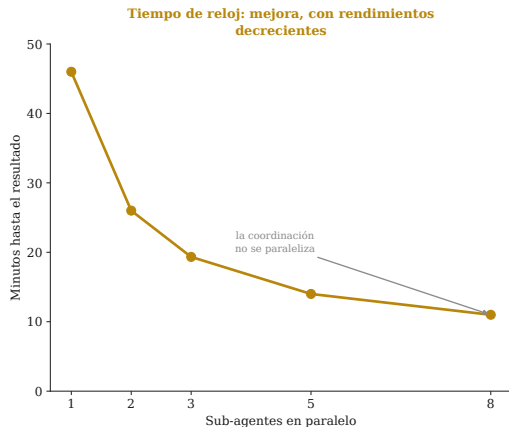
Parte 3 / 6

Sistemas multi-agente



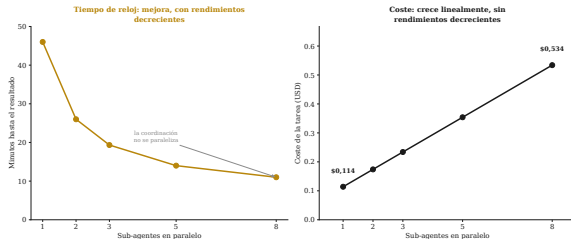
Multi-agente compra tiempo de reloj con tokens.

Multi-agente compra tiempo de reloj con tokens.



Por qué el coste crece y el tiempo no baja igual.

Multi-agente compra tiempo de reloj con tokens.



El tiempo tiene suelo. Repartir el trabajo entre más agentes lo acelera, pero coordinar, repartir y juntar los resultados no se paraleliza. A partir de cierto punto, un agente más apenas aporta.

El coste no tiene techo. Cada sub-agente arranca con su propio contexto base: instrucciones, herramientas, el encargo. Ese contexto se paga entero, tantas veces como sub-agentes lancéis.

EJEMPLO A MANO: Ocho sub-agentes

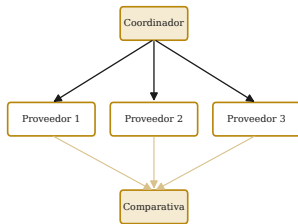
Con 8.000 tokens de contexto base por agente, ocho sub-agentes pagan 64.000 tokens sólo en arrancar, antes de hacer nada útil.

IDEA CLAVE. Lanzad sub-agentes cuando el trabajo sea de verdad independiente y voluminoso; nunca para partir una tarea pequeña.

Tres patrones que compensan el sobrecoste.

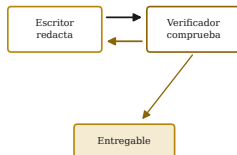
Tres patrones que compensan el sobrecoste.

Abanico



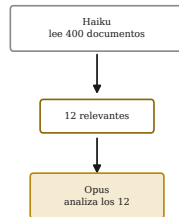
Trabajo independiente que se junta al final.

Escritor y verificador



El verificador llega con contexto limpio y ve lo que el escritor no ve.

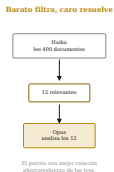
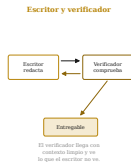
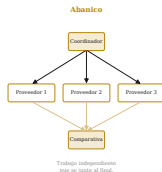
Barato filtra, caro resuelve



El patrón con mejor relación ahorro/esfuerzo de los tres.

Cuándo multi-agente merece la pena, y cuándo no.

Tres patrones que compensan el sobre coste.



Sí, cuando el trabajo se abre en abanico. Analizar 30 ofertas de proveedor, revisar 50 ficheros, investigar 12 competidores. Cada rama es independiente y voluminosa.

Sí, para separar escribir de revisar. El verificador llega con el contexto limpio y detecta lo que el escritor ya no ve, igual que un revisor humano.

Sí, para filtrar barato antes de analizar caro. Haiku descarta 388 documentos de 400; Opus analiza los 12 que quedan. Es el patrón de mejor relación ahorro/esfuerzo.

No, para una tarea secuencial. Si el paso 2 necesita el resultado del paso 1, no hay nada que paralelizar: estaréis pagando contextos de más para hacer lo mismo.

IDEA CLAVE. Regla práctica: si no sabéis enumerar las ramas independientes antes de lanzar, no es un caso multi-agente.

Parte 4 / 6

Adaptar el modelo: SFT, LoRA y RLHF



Tres formas de cambiar el modelo, y qué hace falta para cada una.

Tres formas de cambiar el modelo, y qué hace falta para cada una.

SFT	LoRA	RLHF / DPO
<i>Supervised Fine-Tuning</i>	<i>Low-Rank Adaptation</i>	<i>Aprendizaje por preferencias</i>
Le enseñáis con ejemplos de «pregunta → respuesta correcta».	SFT, pero congelando el modelo y entrenando matrices diminutas al lado.	Le dais pares «esta respuesta mejor que aquella» y aprende la preferencia.
Necesita: Miles de ejemplos etiquetados.	Necesita: Cientos o miles de ejemplos.	Necesita: Comparaciones humanas, caras.
Mueve: Cambia el estilo y el formato.	Mueve: Lo mismo que SFT por el 1 % del coste.	Mueve: Cambia el criterio, no sólo el estilo.

Ninguna de las tres le mete conocimiento nuevo de forma fiable. Para eso está RAG.

Qué cambia de verdad cada técnica.

Tres formas de cambiar el modelo, y qué hace falta para cada una.

SFT	LoRA	RLHF / DPO
<i>Supervised Fine-Tuning</i>	<i>Low-Rank Adaptation</i>	<i>Aprendizaje por preferencias</i>
Le enseñáis con ejemplos de «pregunta → respuesta correcta».	SFT, pero congelando el modelo y entrenando matrices diminutas al lado.	Le dais pares «esta respuesta mejor que aquella» y aprende la preferencia.
Necesita: Miles de ejemplos etiquetados.	Necesita: Cientos o miles de ejemplos.	Necesita: Comparaciones humanas, caras.
Mueve: Cambia el estilo y el formato.	Mueve: Lo mismo que SFT por el 1 % del coste.	Mueve: Cambia el criterio, no sólo el estilo.

Ninguna de las tres le mete conocimiento nuevo de forma fiable. Para eso está RAG.

SFT. Le dais pares «entrada, salida correcta» y ajustáis los pesos para que reproduzca ese estilo. Sirve para formato, tono y estructura.

LoRA. El mismo entrenamiento, congelando el modelo y aprendiendo sólo dos matrices pequeñas. Mismo efecto, una fracción del coste.

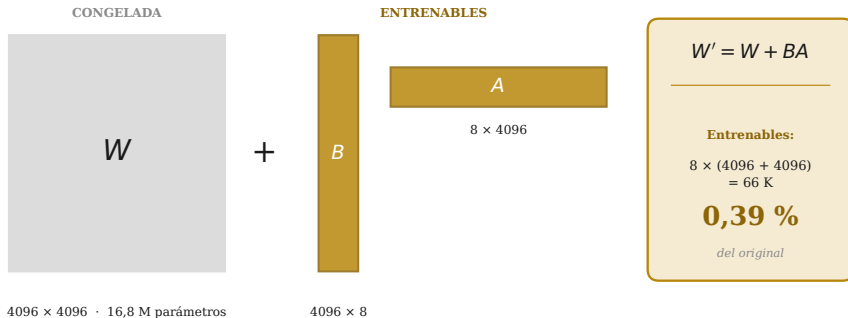
RLHF y DPO. Le dais pares «esta respuesta es mejor que aquella». Aprende una preferencia, no un formato. Es como se enseña a un modelo a ser útil y prudente.

Lo que ninguna hace bien. Meter conocimiento nuevo. Si entrenáis con vuestra normativa, el modelo aprenderá a *sonar* a vuestra normativa y seguirá inventando artículos.

IDEA CLAVE. Conocimiento nuevo → RAG. Estilo y formato → LoRA. Criterio → preferencias.

LoRA: congelar la matriz grande y entrenar dos flacas al lado.

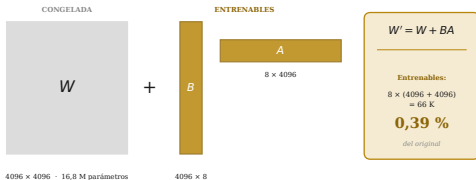
LoRA: congelar la matriz grande y entrenar dos flacas al lado.



Entrenáis una fracción mínima de los parámetros y obtenéis casi todo el efecto. Por eso LoRA cabe en una sola GPU.

La cuenta de LoRA, término a término.

LoRA: congelar la matriz grande y entrenar dos flacas al lado.



Entrenáis una fracción mínima de los parámetros y obtenéis casi todo el efecto. Por eso LoRA cabe en una sola GPU.

$$W' = W + BA$$

W matriz original, $d \times k$, congelada

B matriz entrenable, $d \times r$

A matriz entrenable, $r \times k$

r el rango, típicamente entre 4 y 64

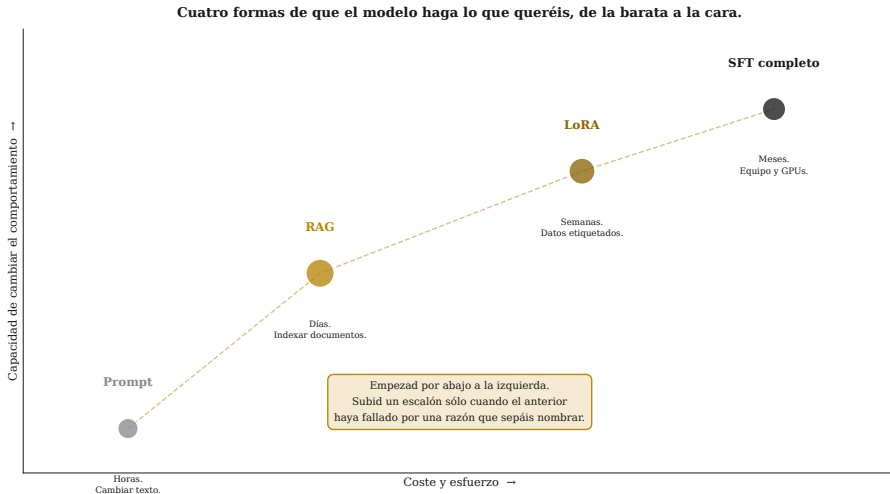
La intuición. El ajuste que necesitáis es mucho más simple que el modelo entero. Cabe en el producto de dos matrices finas.

EJEMPLO A MANO: Con $d = k = 4096$ y $r = 8$

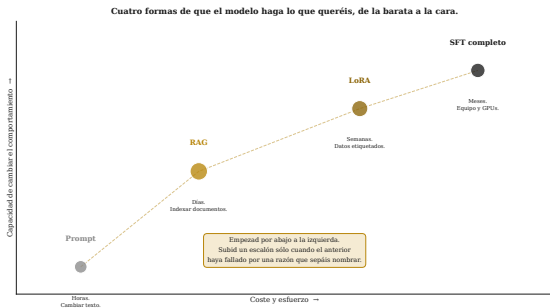
La original tiene $4096^2 \approx 16,8 \text{ M parámetros}$. Las dos flacas, $8 \times (4096 + 4096) = 65,536$. Entrenáis el 0,39 %.

Tres consecuencias prácticas. Cabe en una sola GPU; el resultado pesa megabytes y no gigabytes; podéis tener varios adaptadores y cambiarlos según el cliente.

Cuatro formas de que el modelo haga lo que queréis, de la barata a la cara.



El árbol de decisión, en serio.



¿Le falta información? → RAG. Siempre. Entrenar para meter datos es caro, lento y poco fiable.

¿Responde bien pero con el formato equivocado? → Primero el prompt, con tres o cuatro ejemplos. Si con eso no basta, LoRA.

¿Elige mal entre dos respuestas igual de correctas? → Preferencias (DPO). Es el único caso donde RLHF y sus parientes son la respuesta.

¿Necesitáis que no salga de vuestra red? → Modelo libre, y entonces LoRA sobre él se vuelve natural.

IDEA CLAVE. En el proyecto final, justificad por qué NO habéis elegido el escalón anterior. Esa es la pregunta que os haré en la defensa.

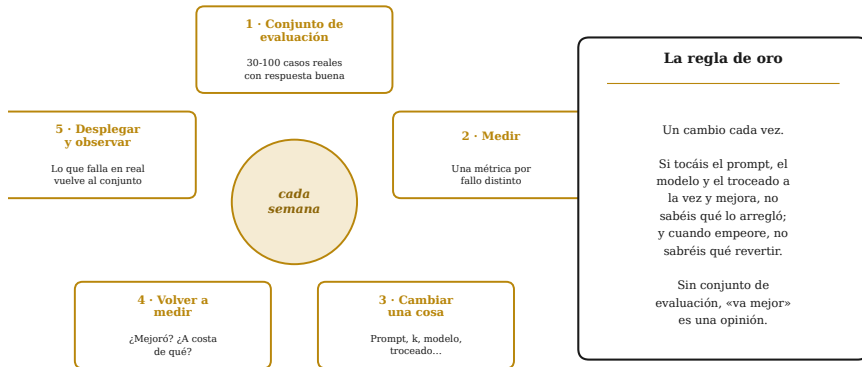
Parte 5 / 6

Loop engineering



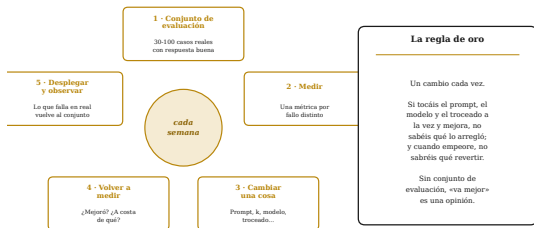
El ciclo que convierte una demo en un producto.

Loop engineering: el ciclo que convierte una demo en un producto.



Cómo se monta el conjunto de evaluación.

Loop engineering: el ciclo que convierte una demo en un producto.



Tamaño. Entre 30 y 100 casos. Menos no distingue señal de ruido; más no os lo vais a mantener.

Procedencia. Preguntas reales de usuarios reales. Las que inventéis en una sala se parecerán a lo que el sistema ya hace bien.

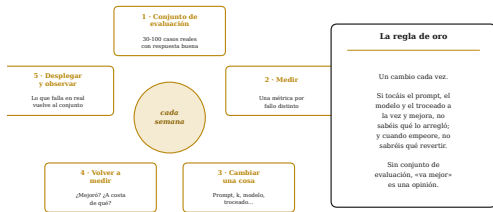
Qué guardáis por caso. La pregunta, la respuesta que consideráis buena, y el documento o dato del que debería salir.

Quién decide la respuesta buena. Alguien del negocio, no del equipo técnico. Si no hay nadie que firme cuál es la respuesta correcta, el proyecto no está listo para empezar.

IDEA CLAVE. Lo que falle en producción se añade al conjunto. Así el conjunto mejora con el tiempo en vez de envejecer.

Las trampas que hacen que un ciclo de evaluación no sirva.

Loop engineering: el ciclo que convierte una demo en un producto.



Cambiar varias cosas a la vez. Si tocáis prompt, modelo y troceado y mejora, no sabéis qué lo arregló; cuando empeore, no sabréis qué revertir.

Evaluar con los casos que usasteis para ajustar. Es el sobreajuste de toda la vida, con otro nombre. Reservad una parte que no se toca.

Usar un modelo como juez sin validarlo. Un LLM puede puntuar respuestas, y es cómodo. Antes comprobad que su puntuación coincide con la de una persona en una muestra.

Medir sólo la media. Un sistema con 85 % de acierto y fallos catastróficos en el 2 % restante es peor para el negocio que uno con 80 % y fallos suaves.

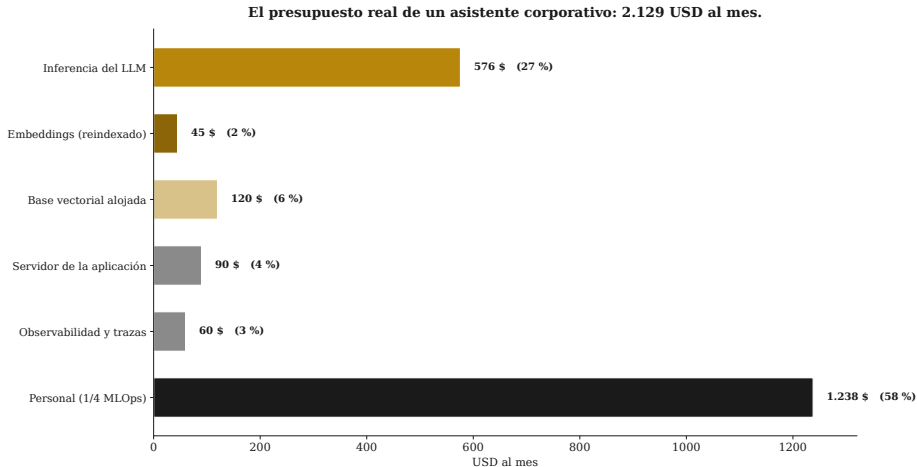
IDEA CLAVE. Observabilidad: guardad cada traza con su coste en tokens. Sin eso, el primer susto en la factura os pillará sin datos.

Parte 6 / 6

El presupuesto completo

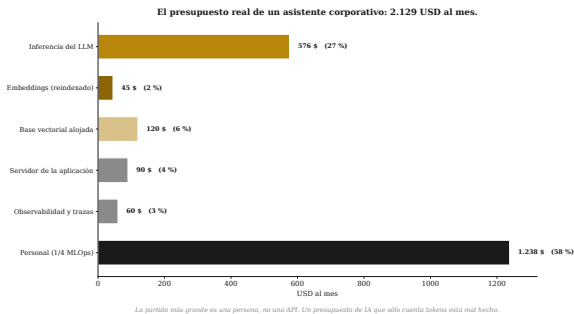


El presupuesto real de un asistente corporativo.



La partida más grande es una persona, no una API. Un presupuesto de IA que sólo cuenta tokens está mal hecho.

Cómo se construye esa tabla para vuestro caso.



Paso 1. Estimad una consulta típica: tokens de entrada, de salida y fracción cacheable. Medidlo, no lo adivinéis.

Paso 2. Multiplicad por vuestra previsión de consultas al mes. Ese número sale del plan de negocio.

Paso 3. Añadid lo que no son tokens: base vectorial, servidor, observabilidad, reindexado.

Paso 4. Añadid personas. Es la partida más grande y la que todos los grupos olvidan.

Paso 5. Dividid entre usuarios activos. Ese es el número que el inversor quiere oír.

IDEA CLAVE. Usad `calculadora_costes.py` del repositorio: hace los cinco pasos y cita la fecha de los precios.

Las tres preguntas que decide vuestro modelo de costes.

Pregunta del inversor	Qué está midiendo	Dónde lo sacáis
¿Cuánto cuesta atender a un usuario al mes?	Si el precio que cobráis deja margen.	Coste total $\div$ usuarios activos.
¿Qué pasa si crecéis $10\times$?	Si vuestro coste es lineal o se dispara.	Repetir el cálculo con $N \times 10$.
¿Y si el proveedor sube el precio?	Vuestra dependencia de un tercero.	Rehacer con el precio $\times 1,5$.
¿Cuánto del coste es fijo?	Cuánto perdéis en un mes malo.	Servidor + personal $\div$ total.
¿Cuándo os compensa el servidor propio?	Si habéis pensado el largo plazo.	Punto de equilibrio del Tema A.

IDEA CLAVE. Un grupo que llegue al pitch con estas cinco respuestas calculadas tiene media defensa hecha.

Lo que os lleváis de este tema.

Cinco sistemas que se construyen encima del agente.



Coste y esfuerzo crecientes → probad siempre de izquierda a derecha

Concepto 1. RAG da información; MCP da acciones; multi-agente da capacidad; la adaptación da criterio. Elegid por lo que falta, no por lo que suena mejor.

Concepto 2. Un RAG se mide con dos números: $\text{recall}@k$ para el buscador y faithfulness para el generador. Fallan por motivos distintos y se arreglan en sitios distintos.

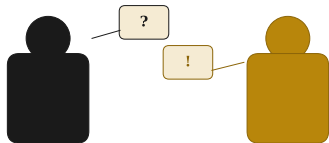
Concepto 3. MCP estandariza la conexión con vuestros sistemas. La combinación peligrosa es leer datos no fiables y poder actuar sobre el mundo.

Concepto 4. LoRA entrena el 0,39 % de los parámetros y consigue casi todo el efecto. Ninguna adaptación mete conocimiento nuevo de forma fiable.

Concepto 5. Sin conjunto de evaluación, «va mejor» es una opinión.

Para el proyecto. El presupuesto completo con las cinco respuestas de la tabla anterior.

Pausa para debatir: ¿a quién le toca responder cuando el RAG se equivoca?



El caso. Vuestro asistente de RR.HH. dice a un empleado que le corresponden 15 días de permiso. Eran 5. El empleado se los coge.

Posición A. Responde la empresa: puso la herramienta, y el empleado hizo lo que le dijeron sus sistemas internos.

Posición B. Responde el empleado: el aviso decía «verifique con RR.HH.», y la cita al convenio estaba ahí.

La pregunta de verdad. ¿Cambia vuestra respuesta si el sistema citaba el artículo correcto y lo resumió mal?

Reglas. Cinco minutos, dos turnos por bando. Yo modero y no participo.

Gracias.

¿Preguntas?

RAG **MCP** **LoRA**

Eduardo C. Garrido-Merchán

ecgarrido@comillas.edu



Hugging Face